

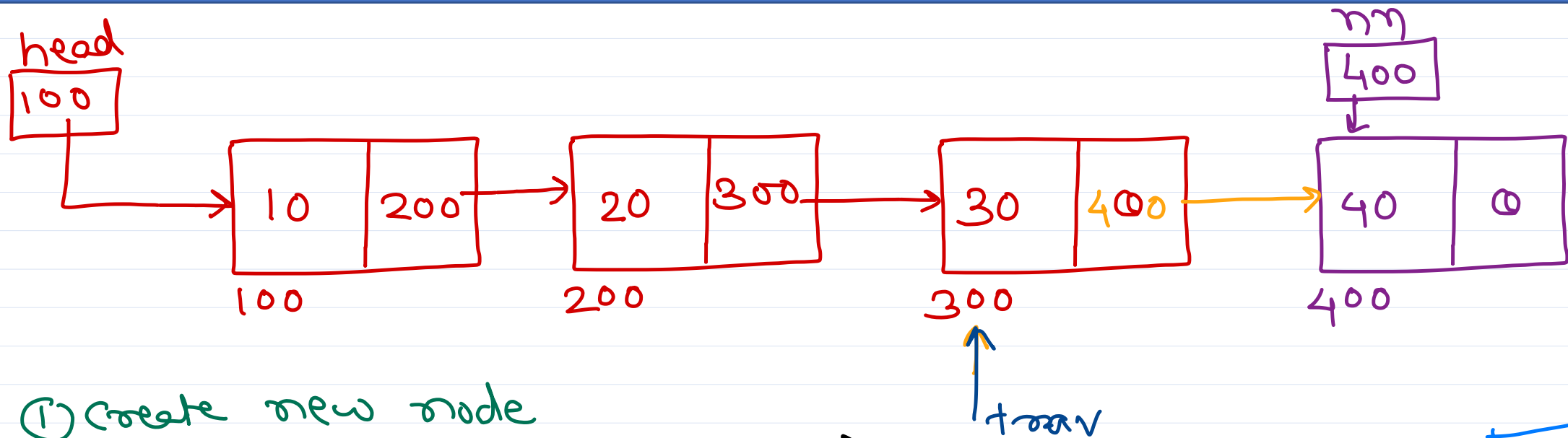


Sunbeam Infotech

Exploring new ideas, Reaching new heights!



Add Last



① Create new node

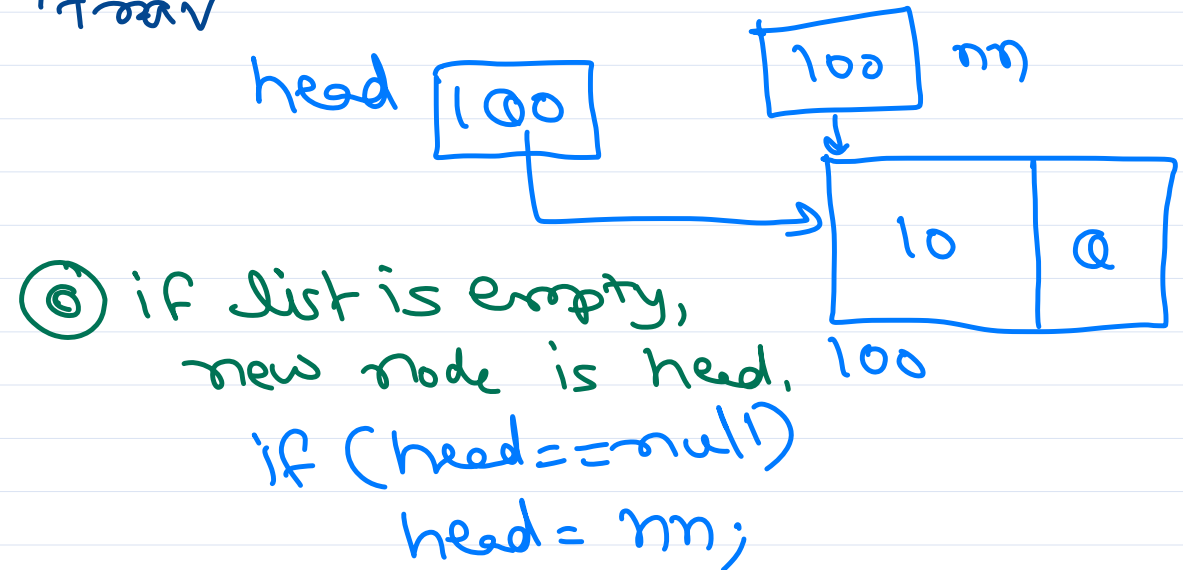
```
ListNode nn = new ListNode(v);
```

② traverse till last node

```
ListNode trav = head;  
while (trav.next != null)  
    trav = trav.next;
```

③ attach new node to next of trav

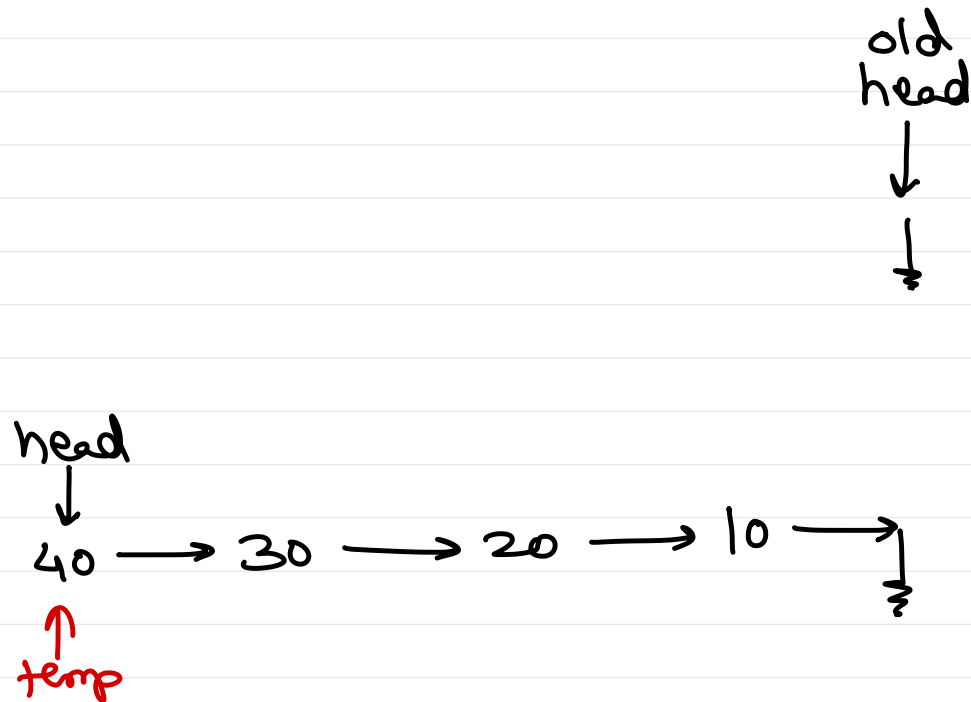
```
trav.next = nn;
```



③ if list is empty,
new node is head.
if (head == null)
 head = nn;



Reverse given list



old
head
↓
↓

consider cur list as old list

oh = h;

and new list as empty list

h = null;

while (oh != null) {

delete first node from old list

t = oh;

oh = oh.next;

add that node at start of new list

t.next = h;

h = t;

} repeat for all node in old list

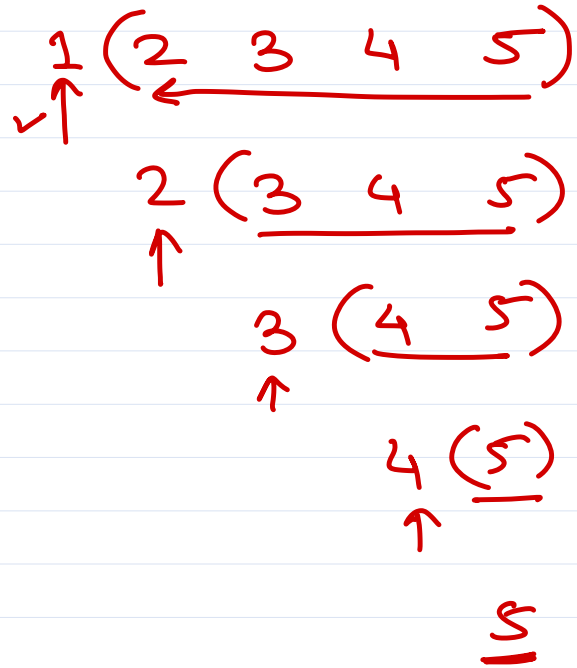


Recursion

print 1 to 5 in rev using recursion.

recursion

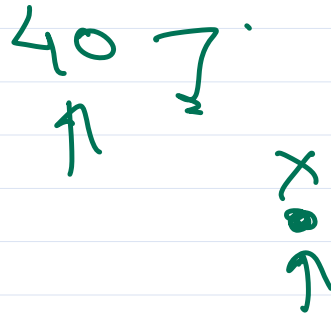
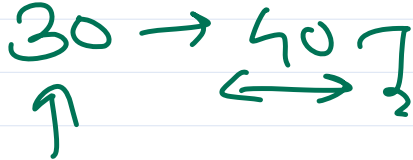
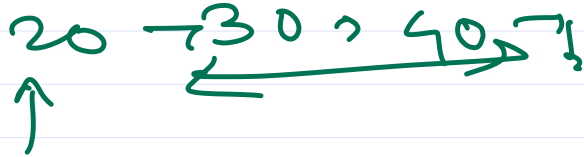
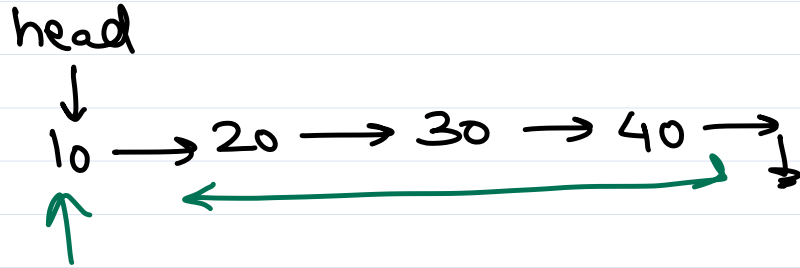
- ① explain process in terms of itself.
- ② identify the end condition.



```
void print(int n) {  
    if (n == 5) {  
        cout << n;  
        return;  
    }  
    print(n+1);  
    cout << n;  
}
```



Display list in reverse order



Approach 1: $\rightarrow O(n^2)$

- ① Count num of nodes in list $\rightarrow n$
- ② traverse till n th node & disp it.
- ③ decrement n . repeat step 2.

Approach 2: $\rightarrow O(n)$

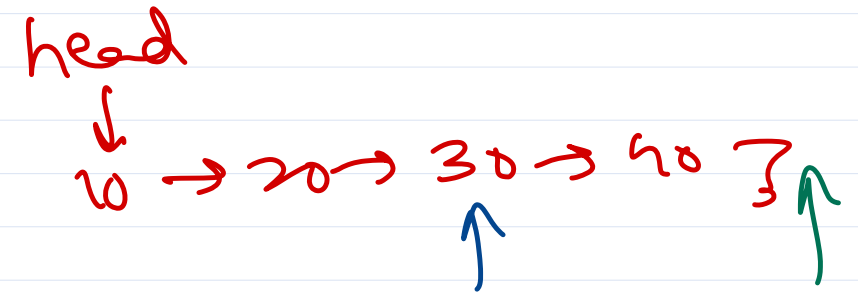
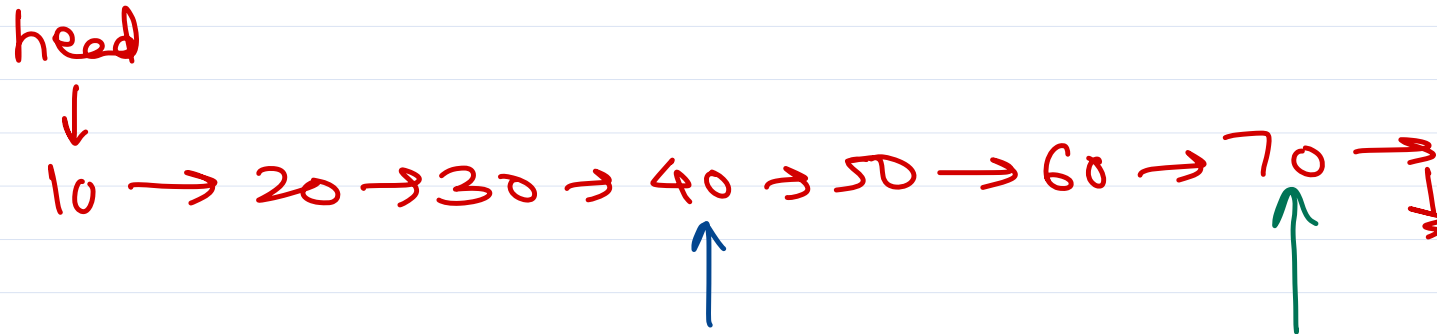
```
void printRev(Node cur) {  
    if (cur == null)  
        return;
```

```
    printRev(cur.next);  
    cout << cur.val;
```

```
}
```



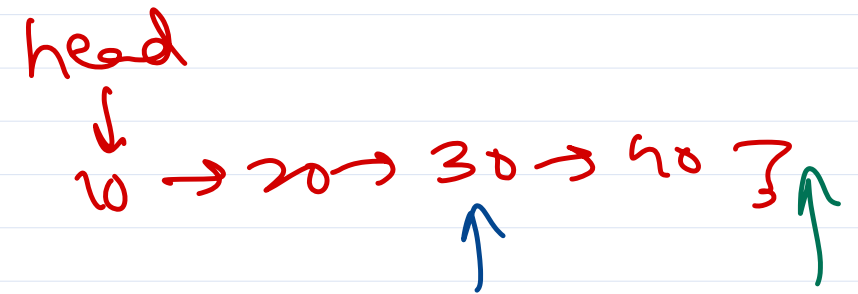
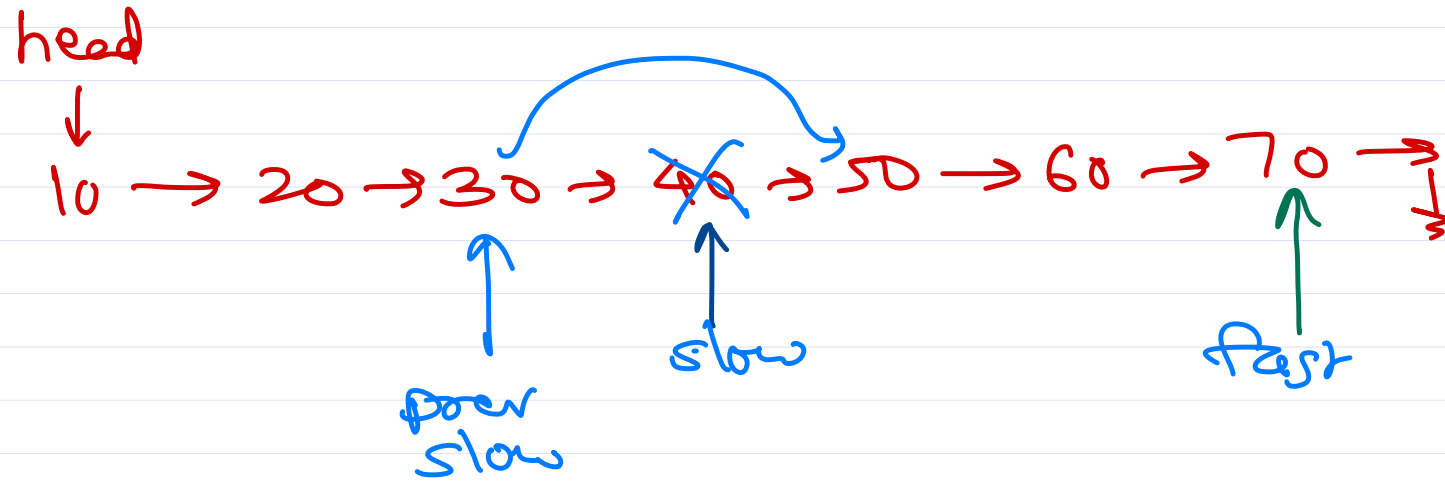
Find Middle



```
slow = head;  
fast = head;  
while(fast != null && fast.next != null) {  
    slow = slow.next;  
    fast = fast.next.next;  
}
```



Delete Middle





Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

